

S3 – Assignment #2: From MIL to PIL

Giovanni Richero, 280245

1 System description

This assignment focuses on the design and implementation of a Proportional–Derivative (PD) attitude controller for a CubeSat, shown in Fig. 1, operating in a low Earth orbit similar to that of the *Hubble Space Telescope* (see more information here). During its nominal mission, the CubeSat maintains a **stable nadir-pointing attitude**, with its **body $+x$ axis directed toward the Earth**.

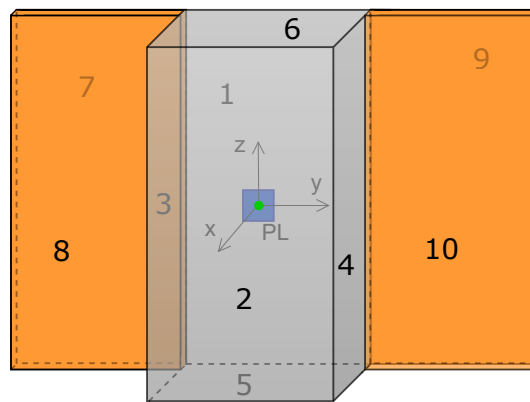


Figure 1: CubeSat geometry and body reference frame.

The attitude dynamics of the CubeSat are simulated using a protected model provided to the students. This model has been developed in **MATLAB & Simulink 2024b** and can be found in the folder `satModel_2ndAssignment_prot.`

The protected model consists of two main modules: AOCS module and Physical module.

1.1 AOCS module

The Attitude and Orbit Control Subsystem (AOCS) module embeds the following features:

- **Attitude sensors**, which include one magnetometer, one Sun sensor, one Earth horizon sensor, and one gyroscope.
- **Attitude actuators**, which include three reaction wheels and three magnetorquers.
- **Attitude determination**, which provides the estimated CubeSat orientation by elaborating sensor's data.

1.2 Physical module

The Physical module computes the following main orbital disturbances and external fluxes:

- **Disturbance torques**, due to gravity gradient, Earth's magnetic field, and solar radiation pressure.

- **Energy balance and heat flow rates**, which considers direct solar flux, Earth's albedo, and infrared emission; it also includes an eclipse model to capture shadowing effects.
- **Orbit propagator**, which computes the evolution of the CubeSat state throughout the simulation, so providing the conditions required for both dynamics and control analysis.

An overview of the protected model output to be used is given in Figure 2. The total simulation time is 3000s; the solver is `ode5` with a fixed-time step of 0.1s. The mission has entered the pointing phase, taking place immediately after the spacecraft completed its slew maneuver.

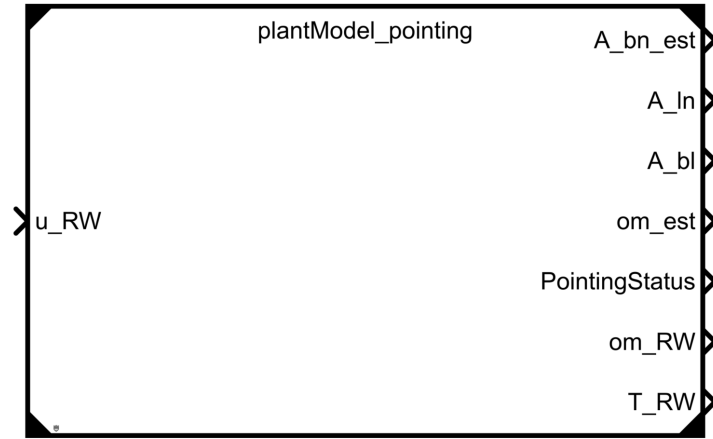


Figure 2: Protected Model overview.

In the mission context, the Earth-Centered Inertial (ECI) frame is the defined inertial frame and the body frame is defined as shown in Figure 1. Letting $\mathbf{r}$ be the position vector, $\mathbf{v}$ be the velocity vector, and $\mathbf{h} = \mathbf{r} \times \mathbf{v}$ be the angular momentum vector, the Local-Vertical-Local-Horizontal (LVLH) frame is defined as:

- X-axis: oriented in the direction $\mathbf{r}$ pointing radially outward from the Earth.
- Y-axis: perpendicular to X and -Z, forming a right-handed coordinate system.
- Z-axis: aligned with $\mathbf{h}$ (normal to the orbital plane).

The protected model receives the single input $\mathbf{u}_{RW}$ as the reaction wheel control torque, and provides the following outputs:

- $\mathbf{A}_{bn_est}$: the estimated direction cosine matrix (DCM) between the body and the inertial reference frames ([3x3]).
- $\mathbf{A}_{In}$: the DCM between the LVLH and the inertial reference frames ([3x3]).
- $\mathbf{A}_{bl}$: the DCM between the body and the LVLH reference frames ([3x3]).
- $\mathbf{om_est}$: the estimated angular velocity of the CubeSat expressed in the body reference frame ([3x1]).
- **PointingStatus**: a trigger that selects the maneuver, between the available ones, to be performed during the simulation. In particular, to perform the pointing maneuver, it has to be equal to three.
- $\mathbf{om_RW}$: the angular velocity of each reaction wheel (in rad/s) ([3x1]). The system contains three reaction wheels in total, each capable of reaching a maximum speed of approximately 470 rad/s.

- $\mathbf{T}_{RW}$: the torque produced by each reaction wheel (in N·m) ([3x1]).

For clarity, the following notation is adopted for all DCMs: a matrix $\mathbf{A}_{xy}$ denotes the DCM that rotates a vector expressed in frame y into the same vector expressed in frame x , i.e.:

$$\mathbf{v}_x = \mathbf{A}_{xy} \mathbf{v}_y \quad (1)$$

In Table 1, the input and outputs of the protected model are listed together with their data type, dimension, and units.

Table 1: Inputs and outputs of the protected model.

Variable Name	Data Type	Dimension	Units
$\mathbf{A}_{b_{nest}}$	double	[3x3]	-
$\mathbf{A}_{In}$	double	[3x3]	-
$\mathbf{A}_{bl}$	double	[3x3]	-
$\mathbf{om}_{est}$	double	[3x1]	-
PointingStatus	int32	[1]	-
$\mathbf{om}_{RW}$	double	[3x1]	rad/s
$\mathbf{T}_{RW}$	double	[3x1]	N·m
$\mathbf{u}_{RW}$	double	[3x1]	N·m

Note: Whenever you plot a physical quantity, clearly specify the reference frame in which that quantity is expressed.

2 Simulations

2.1 Exercise 1: Model-In-the-Loop (MIL)

A control law for the pointing maneuver of the CubeSat towards the Earth must be implemented for the protected model. The control law shall compute the control torque applied by the reaction wheels for a stable nadir-pointing attitude maneuver, with the $+x$ axis directed towards the Earth. The outputs of the protected model must be used as inputs of the developed control law, while the input of the protected model must be the output of the control law.

Eq. (2) reports the PD control action to be implemented to stabilize and track the desired trajectory. The scalar-last convention for the quaternion representation is assumed.

$$\mathbf{u}_{RW} = -k_1 \boldsymbol{\omega}_{error} - k_2 \text{sign}(\delta q_4) \delta \mathbf{q}_{1:3} + \boldsymbol{\omega}_{body} \times I \boldsymbol{\omega}_{body} + I (A_e(\mathbf{q}) \dot{\boldsymbol{\omega}}_d - [\boldsymbol{\omega}_{error}]^\wedge A_e(\mathbf{q}) \boldsymbol{\omega}_d) \quad (2)$$

where:

- $\mathbf{u}_{RW}$: the control torque vector to be applied by the reaction wheels ([3x1]).
- k_1 and k_2 : positive scalar control gains, tuning the rate damping and attitude correction, respectively.
- $\boldsymbol{\omega}_{error} = \boldsymbol{\omega}_{body} - \boldsymbol{\omega}_d$: the angular velocity error between the current spacecraft angular velocity (body-frame) and the desired angular velocity (body-frame) ([3x1]).
- $\delta \mathbf{q}$: the attitude error quaternion between the current and desired attitude ([4x1]).
- $\text{sign}(\delta q_4)$: the sign of the scalar component of the attitude error quaternion.
- $\boldsymbol{\omega}_{body}$: the angular velocity of the spacecraft expressed in the body frame ([3x1]).
- I : the spacecraft inertia matrix ([3x3]).
- $\boldsymbol{\omega}_d$: the desired angular velocity for the pointing maneuver ([3x1]).
- $A_e(\mathbf{q})$: the attitude error matrix corresponding to the quaternion error $\delta \mathbf{q}$ ([3x3]).
- $[\boldsymbol{\omega}_{error}]^\wedge$: the skew-symmetric matrix associated with the angular velocity error ([3x3]).
- $\dot{\boldsymbol{\omega}}_d$: the angular acceleration of the moving frame ([3x1]).

Table 2 reports the nominal values of the mission to be considered.

Table 2: Nominal parameters for the mission.

Variable Name	Value	Dimension	Units
$\boldsymbol{\omega}_0$, nominal	[0; 0; 0]	[3x1]	rad/s
$\mathbf{q}_0$, nominal	[0.2226, 0.1019, -0.9678, 0.0589]	[1x4]	-
Inertia I	$\begin{bmatrix} 0.058816667 & 0 & 0 \\ 0 & 0.054000375 & 0 \\ 0 & 0 & 0.011250375 \end{bmatrix}$	[3x3]	kg·m ²

The pointing performance will be assessed via the scalar quaternion error defined as:

$$\theta = 2 \arccos(|q_4|) \quad (3)$$

where q_4 is the scalar part of the attitude error quaternion.

Considering the protected model, its outputs, and the required input, perform the following tasks:

1) **Configure the simulation environment**

Using a **MATLAB** script, besides the variables present in the base workspace of the protected model, including the *fixedStepSize* and *simTime*, define the variables $\mathbf{q}_0$, ω_0 , and the spacecraft inertia $\mathbf{I}$ using the values presented in Table 2. Verify all variables have the correct dimensions and, for $\mathbf{q}_0$ and ω_0 , the variable names in Matlab must be *q0* and *om0*, respectively, before running the model.

2) **Implement the PD controller**

Implement the control law in Eq. (2) using a Simulink If Action Subsystem block:

- (a) Inside the Simulink If Action Subsystem, in the "Action Port" block, let the two parameters defined as "held" and "only when execution is resumed", respectively. Add and define the required blocks to implement Eq. (2).
- (b) Configure the subsystem such that is only active when the **PointingStatus** flag is equal to three.
- (c) Define the inputs of the Simulink If Action Subsystem needed to implement Eq. (2), with attention for the selection of the desired angular velocity ω_d and desired attitude matrix $\mathbf{A}_d$ such that the $+x$ axis of the spacecraft points towards the Earth along its orbit.
- (d) Wrap the previously defined Simulink If Action Subsystem (as well as its inputs and outputs) in a dedicated subsystem named **Controller Model** and **connect** it to the protected model.
- (e) Add delay unit blocks to the inputs and outputs of the **Controller Model** subsystem to avoid issues with algebraic loop, choosing the correct sample time.

Show and justify the blocks and logic used to build the subsystem and the PD control law, highlighting the formulas considered.

3) **Implement the pointing performance**

Having the protected model block and after defining the **Controller Model** subsystem, create a new subsystem responsible for implementing the pointing performance defined in Eq. (3). Highlight and justify the blocks used inside this subsystem.

4) **Perform Model-In-the-Loop simulation**

Figure 3 exemplifies the expected subsystem blocks after finishing the configuration. Afterward, simulate for 3000s with an ode5 solver with a fixed-time step of 0.1s. For the mission context, after an initial transient phase, the pointing error shall be below 5 deg after 500 seconds and shall remain below this threshold for the rest of the simulation. Tune the control gains $\mathbf{k}_1$ and $\mathbf{k}_2$ to obtain the desired pointing.

Show and comment on the performance of the implemented control law. In addition to any other plot you might consider useful for the analysis, plot the time history of the pointing error, the quaternion error, the angular velocity error, and the control torque of the reaction wheels.

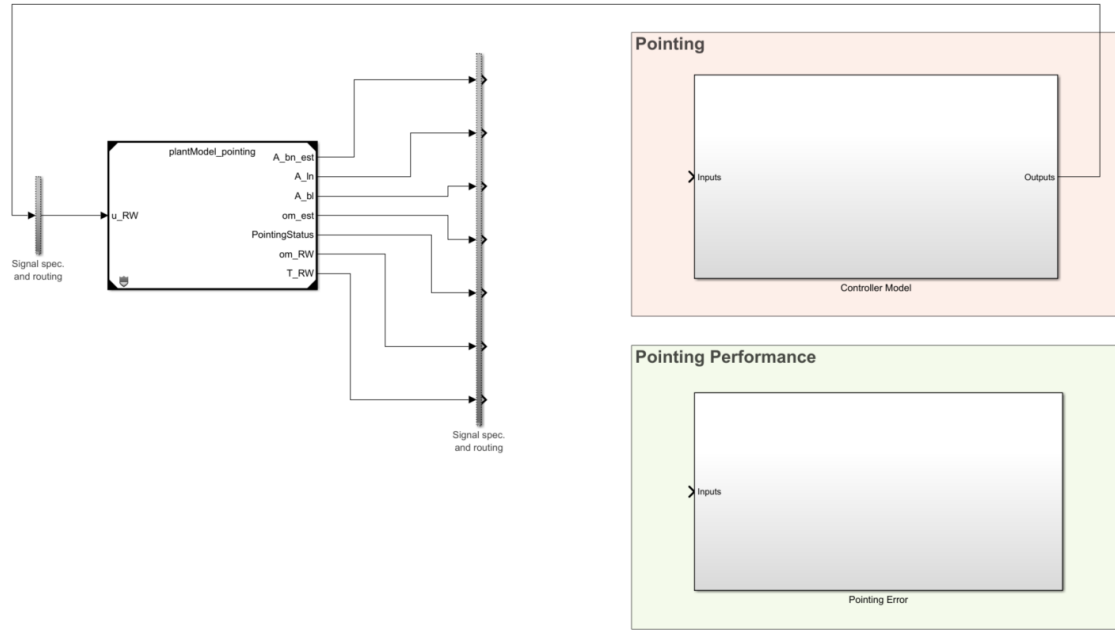


Figure 3: Illustrative example of expected subsystem blocks at MIL level.

To evaluate the robustness of the PD controller with respect to uncertainties in the initial state, perform a **Monte-Carlo analysis with $N = 200$** independent simulations (see lecture 09). In each simulation, the initial angular velocity ω_0 and the initial attitude quaternion $\mathbf{q}_0$ must be perturbed as follows:

- Initial angular velocity perturbation: the nominal angular velocity $\omega_{0, \text{nominal}}$ report in Table 2 is perturbed by adding a zero-mean Gaussian noise term with standard deviation corresponding to $3\sigma = 10 \text{ deg/s}$.
- Initial attitude quaternion perturbation: generated by drawing its four components from a standard normal distribution and normalizing it.

After the Monte-Carlo simulation is completed:

- 5) Produce the following plots: a histogram for each component of the perturbed initial angular velocity ω_0 , a histogram of the euler angles corresponding to each sampled $\mathbf{q}_0$ (obtained using the **MATLAB** function *quat2eul* with rotation sequence 'XYZ'), the time history of the pointing error, the quaternion error, and the angular velocity error for all Monte-Carlo runs.
- 6) Comment on the obtained results. Does the controller maintain acceptable performance across all perturbations? Based on the statistical evidence, is your PD controller robust and validated?

Model-In-the-Loop (MIL)

The nadir-pointing PD controller is implemented in Simulink following the input-output interface defined in Table 1. As shown in Fig. 4, the overall architecture is organized, together with the protected model itself, into three main subsystems: the controller subsystem, responsible for generating the reaction wheel control torque and enforcing stable nadir-pointing, and the

pointing performance subsystem, used to evaluate the accuracy of the achieved attitude pointing.

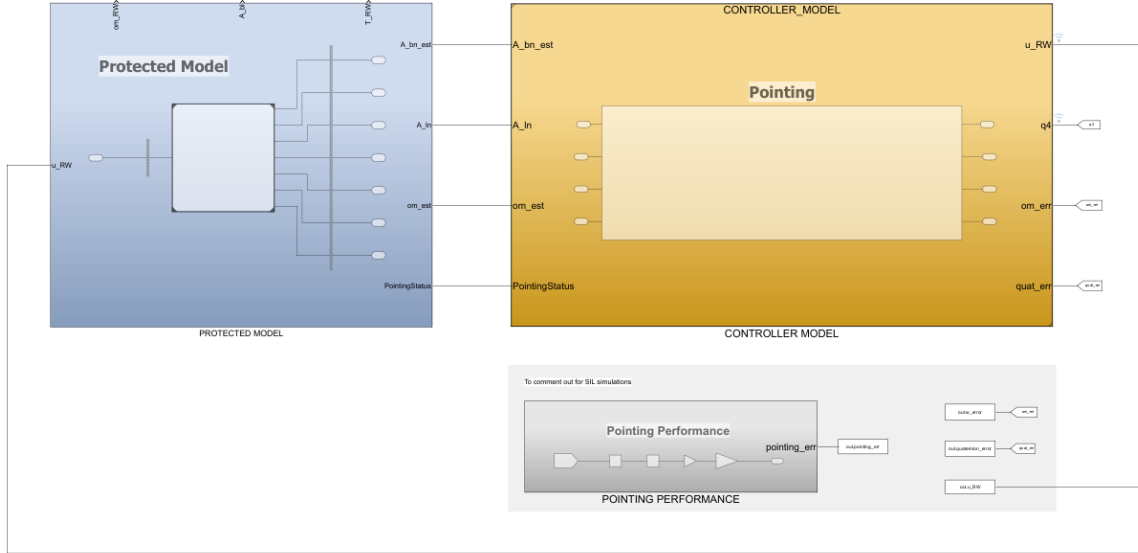


Figure 4: Main subsystems configuration Simulink

The PD control action is implemented in accordance with the following non-linear control law:

$$\mathbf{u}_{RW} = -k_1 \boldsymbol{\omega}_{\text{error}} - k_2 \text{sign}(\delta \mathbf{q}_4) \delta \mathbf{q}_{1:3} + \boldsymbol{\omega}_{\text{body}} \times I \boldsymbol{\omega}_{\text{body}} + I (\mathbf{A}_e(\mathbf{q}) \dot{\boldsymbol{\omega}}_d - [\boldsymbol{\omega}_{\text{error}}]^\wedge \mathbf{A}_e(\mathbf{q}) \boldsymbol{\omega}_d) \quad (4)$$

where the desired angular rate $\boldsymbol{\omega}_d$, the angular rate error $\boldsymbol{\omega}_e$, the attitude error matrix $\mathbf{A}_e$, the attitude error quaternion $\delta \mathbf{q}$, and the estimated body angular rate $\boldsymbol{\omega}_{\text{body}}$ are computed using the output information provided by the protected model.

As presented in Fig. 5, which illustrates the controller model subsystem, all the quantities required by the control law are computed outside the core build-up control logic defined in Eq. (4).

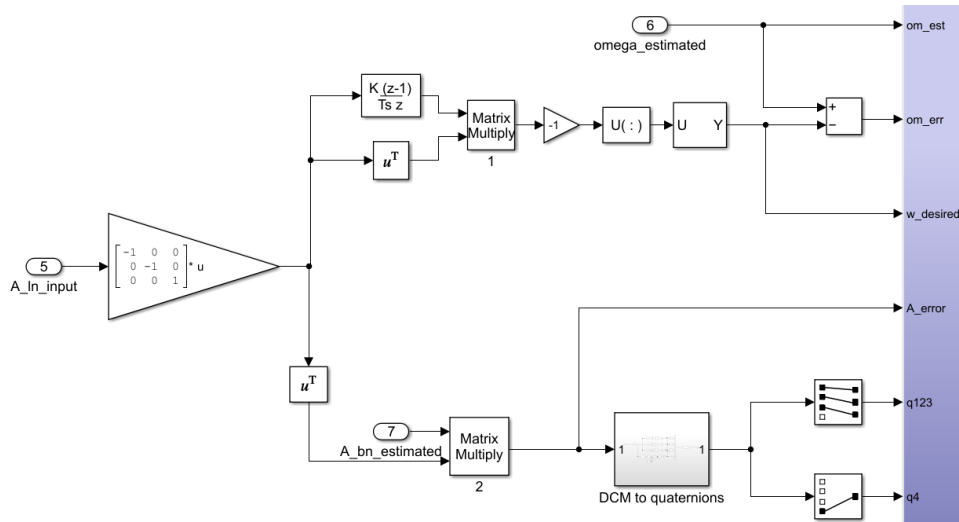


Figure 5: Simulink logic implemented for input pre-calculation

First, the desired attitude is defined in order to compute the attitude error between the current body reference attitude and the target attitude required to achieve stable nadir pointing.

The desired attitude is derived from the Local Vertical Local Horizontal (LVLH) frame $\mathbf{A}_{ln}$, which by definition rotates along the orbit with its $+\hat{x}$ axis aligned with the zenith direction. Since the CubeSat must maintain its $+\hat{x}$ axis aligned with the nadir direction, the desired attitude $\mathbf{A}_d$ is computed as a constant rotation of 180° about the z -axis on the $\mathbf{A}_{ln}$:

$$\mathbf{A}_d = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \mathbf{A}_{ln} \quad (5)$$

The attitude error matrix $\mathbf{A}_e$ is obtained as the relative rotation between the estimated attitude and the desired attitude:

$$\mathbf{A}_e = \mathbf{A}_{b_{nest}} \mathbf{A}_{desired}^T \quad (6)$$

A dedicated Simulink subsystem converts the attitude error matrix $\mathbf{A}_e$ into the error quaternion $\delta \mathbf{q}$, as reported in Fig. 5 and 6.

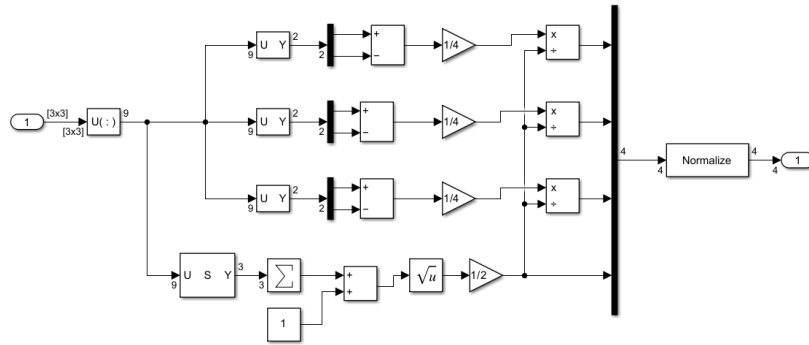


Figure 6: DCM-to-quaternions subsystem

The subsystem utilized follows the standard DCM-to-quaternion formulation, in which the quaternion components are computed as:

$$\begin{aligned} \delta q_1 &= \frac{1}{4 \delta q_4} (A_e(2, 3) - A_e(3, 2)), \\ \delta q_2 &= \frac{1}{4 \delta q_4} (A_e(3, 1) - A_e(1, 3)), \\ \delta q_3 &= \frac{1}{4 \delta q_4} (A_e(1, 2) - A_e(2, 1)), \\ \delta q_4 &= \frac{1}{2} \sqrt{1 + A_e(1, 1) + A_e(2, 2) + A_e(3, 3)}. \end{aligned} \quad (7)$$

In the Simulink implementation, the attitude error matrix $\mathbf{A}_e$ is decomposed using *Selector* blocks to extract the diagonal and off-diagonal elements respectively for δq_4 and for the $\delta q_{1:3}$. Each component is then computed through *Sum/Subtract*, *Gain*, and *Sqrt* blocks according to Eq. (7), while the scalar component δq_4 of the quaternion is enforced to be positive, by convention. Moreover, a *Normalize* block is applied to guarantee $\|\delta \mathbf{q}\| = 1$ at each simulation step, preventing numerical drift.

To further retrieve the desired angular rate $\boldsymbol{\omega}_d$ the time derivative $\dot{\mathbf{A}}_d$ is computed using a *Discrete Derivative* block. From the kinematic identity:

$$\dot{\mathbf{A}}_d = -[\boldsymbol{\omega}_d]^\wedge \mathbf{A}_d \quad [\boldsymbol{\omega}_d]^\wedge = -\dot{\mathbf{A}}_d \mathbf{A}_d^T \quad (8)$$

The steps shown in Eq. (8) to compute and then extract the vector components from the skew-symmetric matrix $(\cdot)^\wedge$ are performed respectively with *Matrix Multiply*, *Reshape*, and *Selector* blocks. The angular rate error $\boldsymbol{\omega}_e$ is then computed as:

$$\boldsymbol{\omega}_{error} = \boldsymbol{\omega}_{est} - \boldsymbol{\omega}_d \quad (9)$$

Control law implementation

With $\delta \mathbf{q}$, ω_e , ω_d available, the remaining terms of the control law defined in Eq. (4) are implemented within a dedicated subsystem of the Controller Model. As shown in Fig. 7 this subsystem is configured as an If Action subsystem and is activated only when the `PointingStatus` flag provided by the protected model is equal to three.

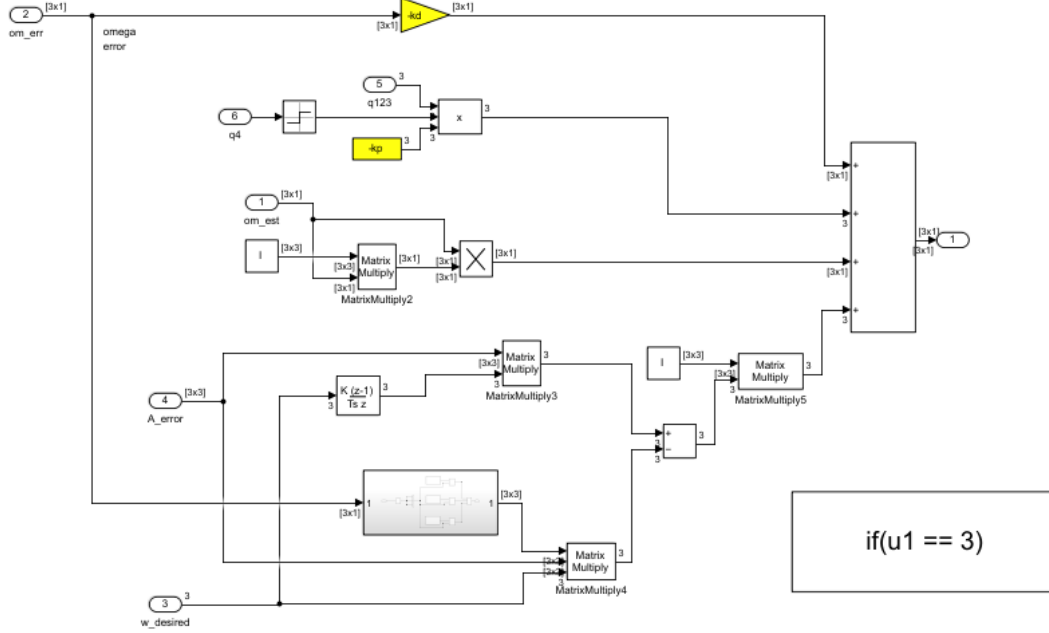


Figure 7: If Action subsystem

The *Selector* blocks extract δq_4 and the *sign* block computes $\text{sign}(\delta q_4)$. While $\dot{\omega}_d$ is far computed with the *Discrete Derivative* block from ω_d . To impose proportional and derivative gains k_1 and k_2 the *Gains* are applied. The *Cross Product* block is needed to compute $[\omega_{\text{body}}]^\wedge I \omega_{\text{body}}$, while *Matrix Multiply* blocks compute:

$$\mathbf{I} \mathbf{A}_e \dot{\omega}_d \quad \text{and} \quad -\mathbf{I} [\omega_e]^\wedge \mathbf{A}_e \omega_d, \quad (10)$$

The *Sum* block, then, is utilized to build-up all the terms together, assembling the final control torque $\mathbf{u}_{RW}$ as shown in Eq. (2). From this assembly the output is returned as feedback loop input to the protected model generating the nadir-pointing PD control law. The *Unit Delay* blocks are placed at the controller outputs to avoid algebraic loops.

Pointing Performance

The pointing performance is assessed via the scalar quaternion error Eq. (3).



Figure 8: Pointing performance subsystem

For this computation, a dedicated subsystem (see Fig. 8) is implemented, taking as input δq_4 . An *Abs* block is first used to compute the absolute value of δq_4 , followed by an *acos* block to evaluate its arccosine. The resulting angle is then multiplied by a factor of 2 using a *Gain* block, in accordance with the definition of the pointing error. Finally, an additional *Gain* block is applied to convert the angle from radians to degrees.

Design of the PD gains

In the small-angle regime, each principal axis can be approximated as a decoupled second-order system, neglecting the gyroscopic coupling term $\boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$. Denoting by θ the attitude error about a principal axis, by τ the applied control torque, and by I the corresponding principal moment of inertia, the linearized closed-loop rotational dynamics can be written as:

$$\begin{cases} I\ddot{\theta} \simeq \tau, \\ \tau = -k_p\theta - k_d\dot{\theta}, \end{cases} \implies \ddot{\theta} + \frac{k_d}{I}\dot{\theta} + \frac{k_p}{I}\theta = 0 \implies \ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = 0. \quad (11)$$

By identity, the control gains follow as $k_p = I\omega_n^2$ and $k_d = 2\zeta I\omega_n$. From which the natural frequency ω_n and elongation ζ are selected knowing that the pointing error must be below 5 deg after 500 s and remain below this threshold thereafter. A conservative way to satisfy this requirement is to enforce that the closed-loop transient is essentially settled within 500 s, considering the settling time T_s as 2% of this maximum allowable time [1]. The damping ratio ζ is selected to avoid oscillations and overshoot in the pointing error and to obtain a reasonably fast response. Values in the range $\zeta \in [0.8, 1]$ are preferred for pointing applications [1]. In this work, $\zeta = 0.9$ and $T_s = 10$ s are considered and so ω_n :

$$\omega_n = \frac{4}{\zeta T_s} \simeq 0.4 \text{ rad/s}. \quad (12)$$

To extend the control to the three-axis attitude dynamics, the proportional and derivative gain vectros are computed as reported in Eq. (13).

$$\mathbf{k}_p = \text{diag}(I_{xx}, I_{yy}, I_{zz})\omega_n^2, \quad \mathbf{k}_d = 2\zeta \text{diag}(I_{xx}, I_{yy}, I_{zz})\omega_n, \quad (13)$$

Nominal case

Accordingly with the initial condition $\boldsymbol{\omega}_0$ and $\mathbf{q}_0$ a simulation regarding the nominal condition is performed over a time horizon of 3000 s using `ode5` with a fixed-step time of 0.1 s.

The controller gains are tuned from the second-order design procedure previously described and then a refined value of ω_n (see Eq. (12)) is selected after trials simulation. The final gains adopted are: $\mathbf{k}_p = [0.0094; 0.0086; 0.0018]$ and $\mathbf{k}_d = [0.0212; 0.0194; 0.0041]$.

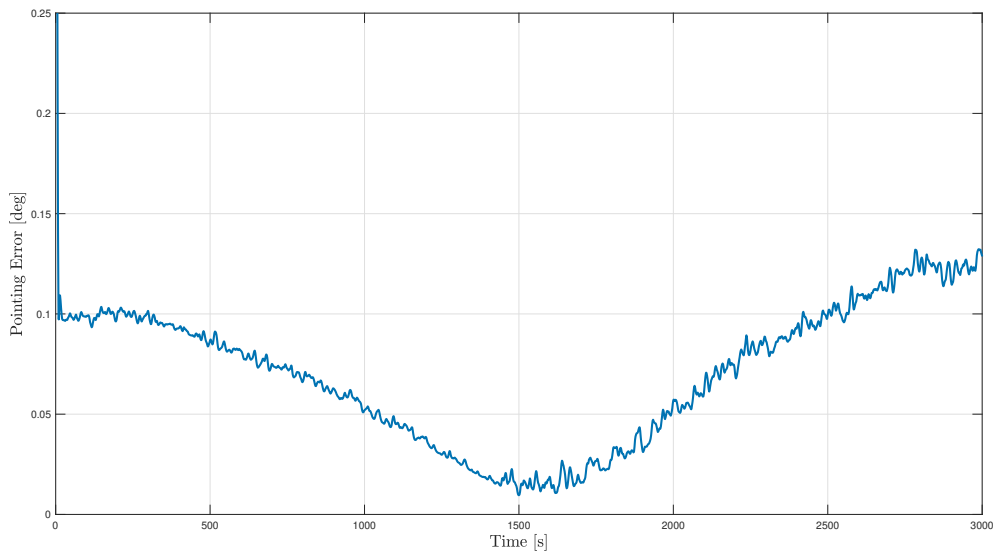


Figure 9: Time history of the pointing error

Fig. 9 shows the time history of the pointing error θ , defined in Eq. (3). After an initial transient, the pointing error decreases below the required threshold of 5 deg well before 500 s and remains consistently below this limit for the remainder of the simulation. These initial nominal conditions represent a reasonable guess that enables fast convergence of the controller; therefore, a Model-In-the-Loop analysis is required to verify that the implemented control law satisfies the pointing performance requirements. The pointing error shows a smooth transient response is observed, with no overshoot exceeding the prescribed bound. This behavior is consistent with the selected damping ratio, chosen to favor a robust damped response to acquire immediately a stable pointing. This choice motivates the subsequent Monte Carlo analysis, which has to be performed to assess the validity of the controller and verify the convergence of the pointing error over a large set of initial conditions.

The convergence behavior is confirmed also from the angular velocity error w_{error} , reported in Fig. 10.

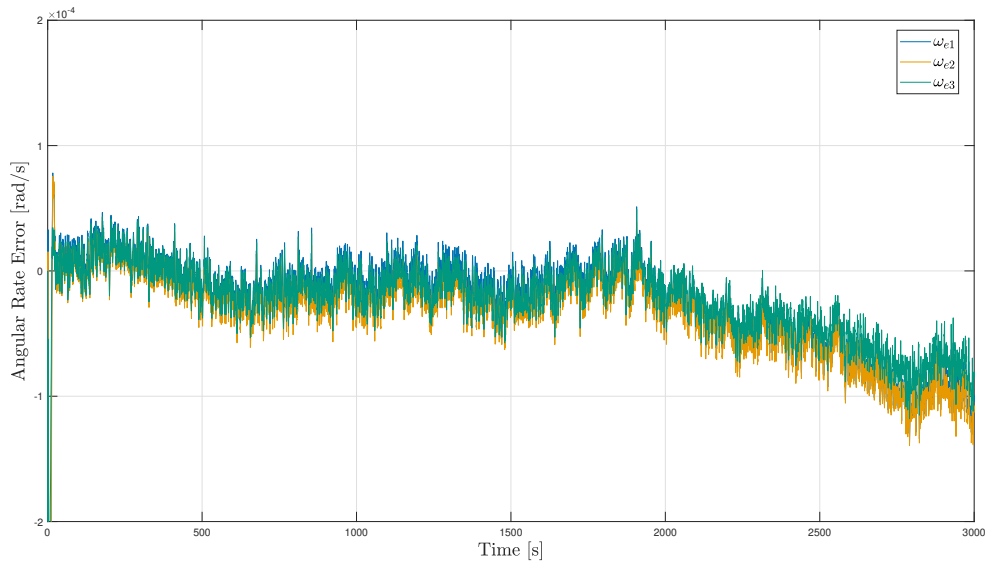


Figure 10: Time history of the angular velocity error expressed in the body frame

After the initial transient, ω_{error} converges to values close to zero, with residual fluctuations on the order of 10^{-4} rad/s. These small residual errors are mainly attributed to the presence of environmental disturbances and sensor noises, coming from the protected model, and to the discrete-time implementation of the control law. The small residual error drift observed between 2000 s and 3000 s is consistent with the periodical variation reported in the time history of the pointing error (Fig. 9).

The control torque generated by the reaction wheels is shown in Fig. 11. The commanded torque remains bounded within values in the order of 10^{-6} and exhibits a smooth noises profile throughout the simulation, without signs of saturation. This indicates that the selected controller gains provide an effective trade-off between pointing accuracy and actuator effort, while respecting the saturation of the reaction wheels.

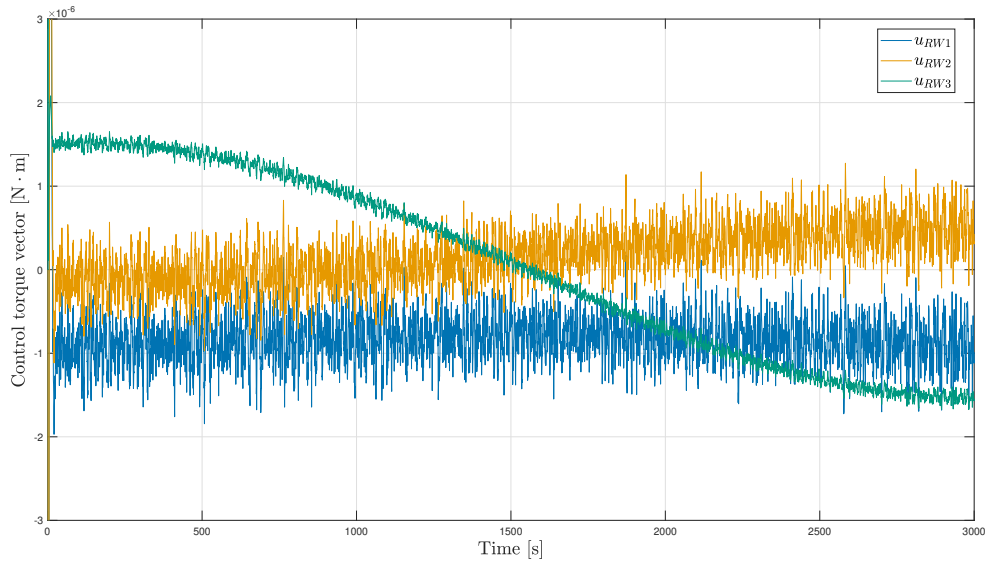


Figure 11: Time history of the control torque of the RWs

Finally, the quaternion error components shown in Fig. 12 confirm stable attitude tracking, with bounded errors of about 10^{-6} between 1 for δq_4 and 10^{-3} between 0 for $\delta q_{1:3}$. The time history of δq_4 reflects the expected behavior dictated by Eq. (3), exhibiting the same overall trend as the pointing error, with opposite maxima and minima.

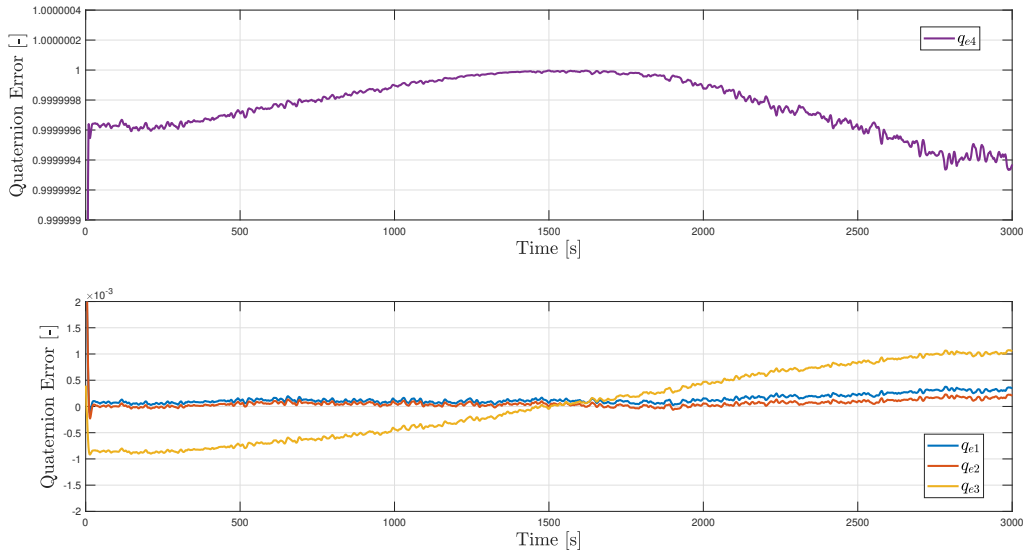


Figure 12: Time history of the quaternion errors

Overall, the nominal simulation demonstrates that the implemented PD control law achieves stable nadir-pointing, satisfies the mission pointing requirements, and operates within acceptable actuator limits.

Monte Carlo analysis

To assess the robustness of the proposed Control Model with respect to uncertainties in the initial conditions, a Monte Carlo analysis is performed. A total of $N = 200$ independent simu-

lations are executed, by randomly perturbing both the initial attitude and the initial angular velocity of the spacecraft

Initial conditions

The nominal initial angular velocity vector $\boldsymbol{\omega}_{0,\text{nominal}}$ is perturbed by adding a zero-mean Gaussian noise term. The standard deviation is selected such that $3\sigma = 10 \text{ deg/s}$ leading to $\sigma = 0.0582 \text{ rad/s}$. The N samples of the perturbed initial angular velocity are generated as

$$\boldsymbol{\omega}_0 = \boldsymbol{\omega}_{0,\text{nominal}} + \sigma \boldsymbol{\eta}, \quad \boldsymbol{\eta} \approx \mathcal{N}(\mathbf{0}). \quad (14)$$

As reported in Fig. 13, in which the histograms of the perturbed initial angular velocity components $\omega_{0,x}$, $\omega_{0,y}$, and $\omega_{0,z}$ are shown, the normal distributions closely match the imposed zero-mean Gaussian profiles.

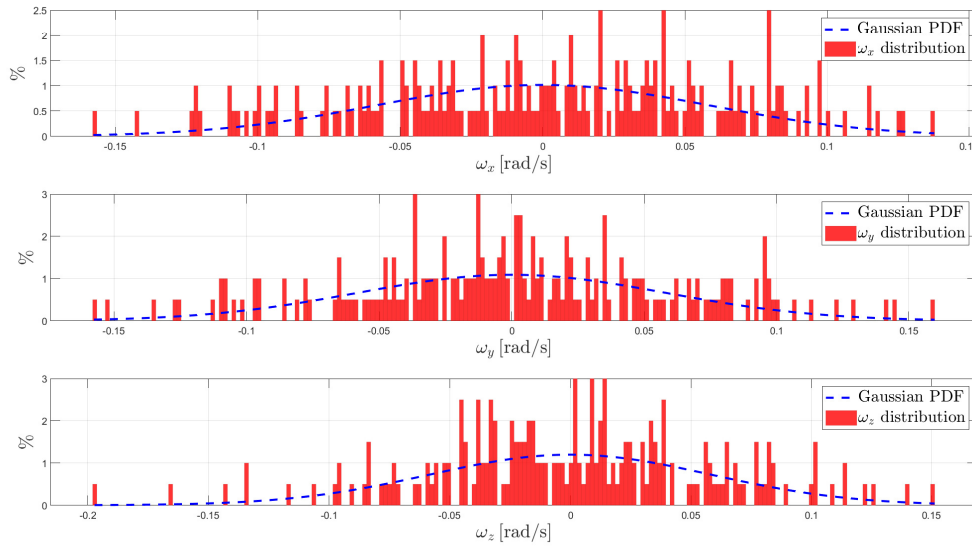


Figure 13: Histogram for each component of the perturbed initial angular velocity $\boldsymbol{\omega}_0$,

The spread of the distributions is consistent with the prescribed σ confirming the correct implementation of the angular rate uncertainty model.

Similarly, the nominal initial attitude quaternion $\mathbf{q}_{0,\text{nominal}}$ is perturbed by sampling its four components from a standard normal distribution and normalizing the resulting vector.

$$\tilde{\mathbf{q}}_0 \approx \mathcal{N}(\mathbf{0}), \quad \mathbf{q}_0 = \frac{\tilde{\mathbf{q}}_0}{\|\tilde{\mathbf{q}}_0\|}. \quad (15)$$

Fig. 14 shows the histograms of the Euler angles (α, β, γ) corresponding to the sampled initial attitude quaternions $\mathbf{q}_0$, obtained through the `quat2eul` function using the XYZ rotation sequence.

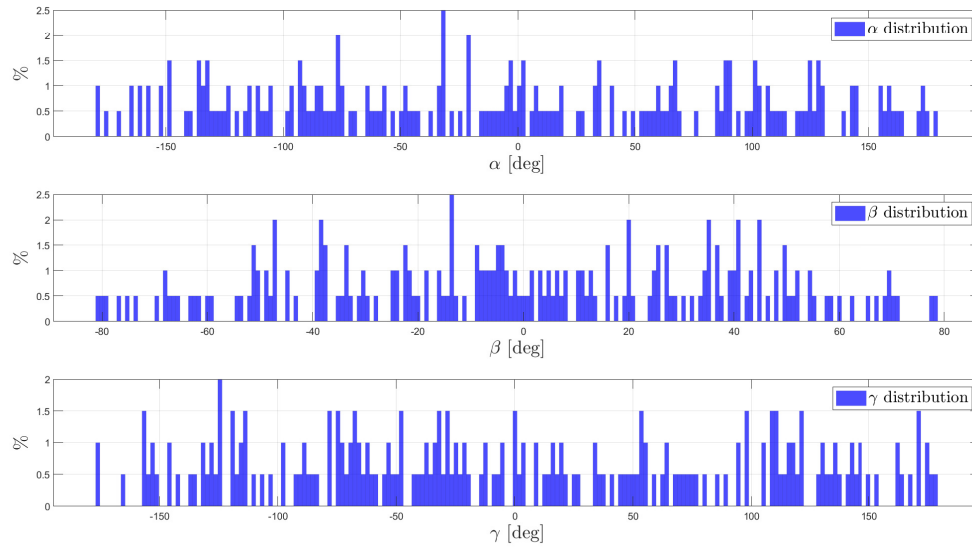


Figure 14: Histogram of the Euler angles corresponding to each sampled $\mathbf{q}_0$,

In this case, the Euler angles do not exhibit a normal distribution, since they are obtained by converting the sampled initial quaternions $\mathbf{q}_0$, while the original normal distribution is not preserved through the nonlinear `quat2eul` function conversion. Nevertheless, the resulting distributions span a wide range of angles, confirming that the Monte Carlo campaign effectively explores a large portion of the attitude initial condition, including large initial misalignments. The controller gains are kept identical to the nominal case, in order to evaluate the closed-loop performance and verify the robustness of the control design.

Results

Here, the results of for each of the 200 Monte Carlo runs are reported. Fig. 15, the time histories of the pointing error, confirms that all the initial sampled conditions lead the pointing error to converge allowing the desired nadir-pointing attitude.

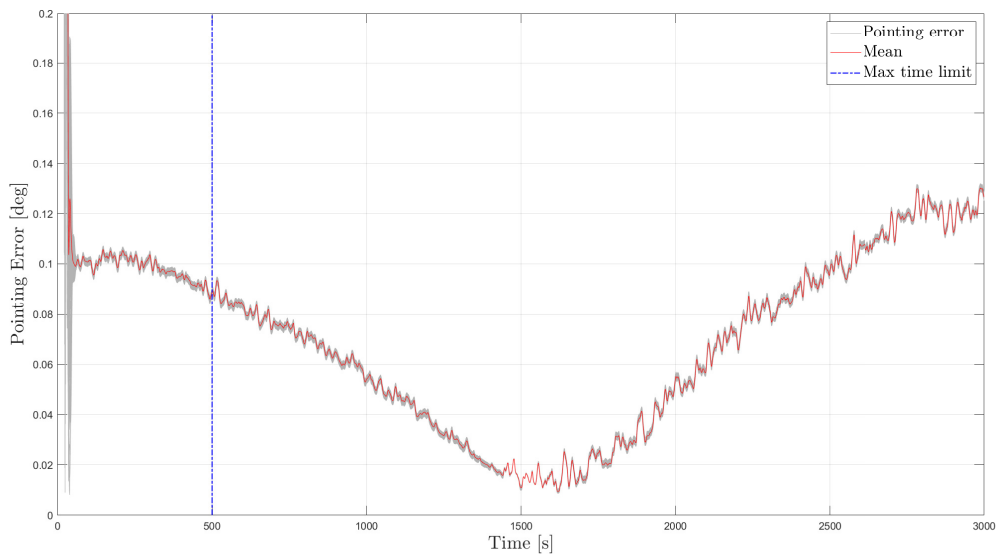


Figure 15: Time histories of the pointing error for 200 Monte-Carlo runs

Fig. 16 shows a zoomed view of the the early phase of the 200 simulations. Although the large dispersion in the initial conditions the pointing errors reach a stable convergence under the 5 deg in less than 40 s, well below the maximum allowable settling time of 500 s.

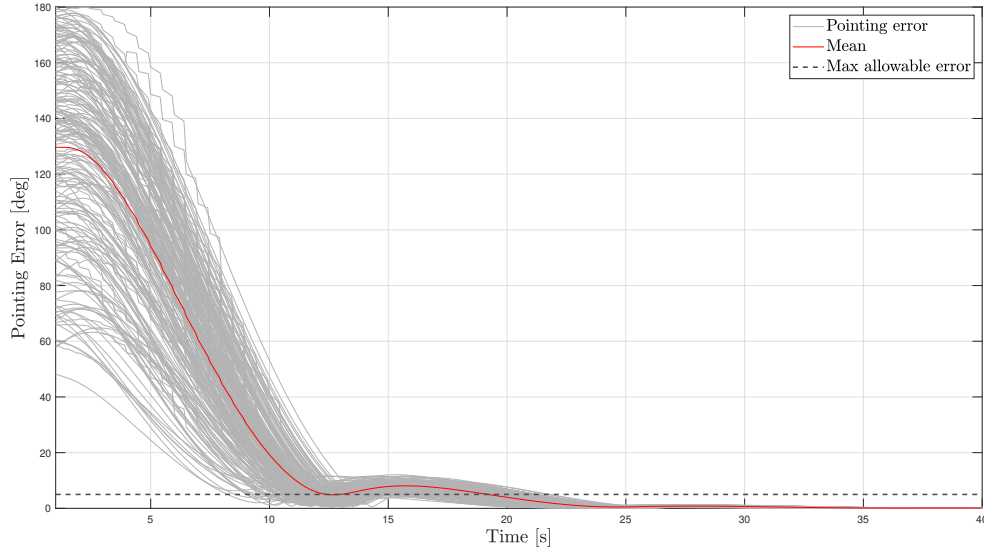


Figure 16: Early simulations zoom of the pointing error for 200 Monte-Carlo runs

This result confirms that the performance requirement is satisfied for all realizations, demonstrating robustness with respect to initial attitude and angular rate uncertainties. The corresponding δq_4 and $\delta q_{1:3}$ histories, reported in Fig. 17, exhibit a consistent convergence trend across all simulations.

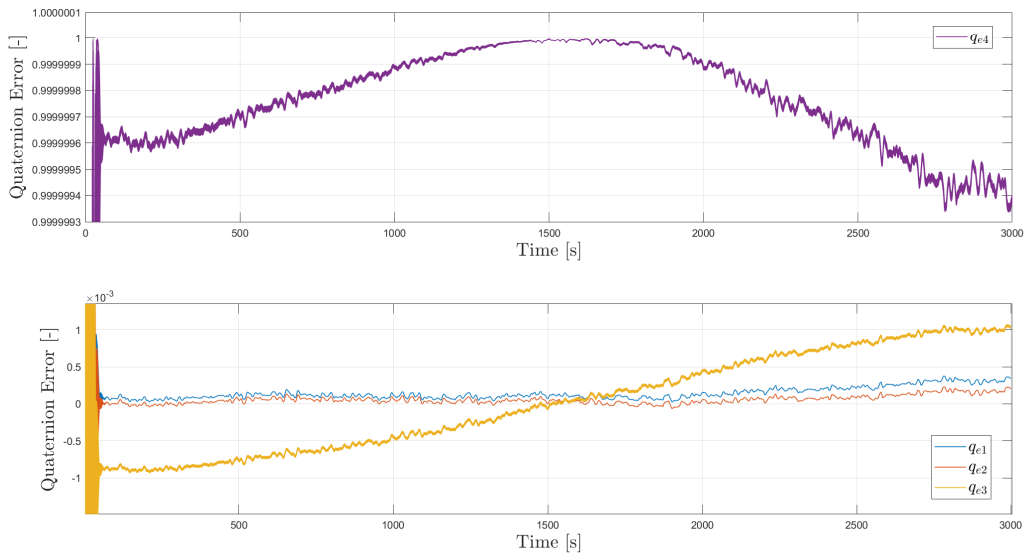


Figure 17: Time histories of the quaternion error for 200 Monte-Carlo runs

After the initial alignment phase, the quaternion error remains close to the trend of the nominal case Fig. 12, indicating stable attitude pointing with no divergence.

As for the quaternion error the initial angular rates ω_e (Fig. 18) span a wide range due to the imposed sampled initial conditions, the PD controller successfully damps the rotational motion

in all cases. After convergence, the angular rates remain confined within a narrow band around zero, with no evidence of instability or sustained oscillations.

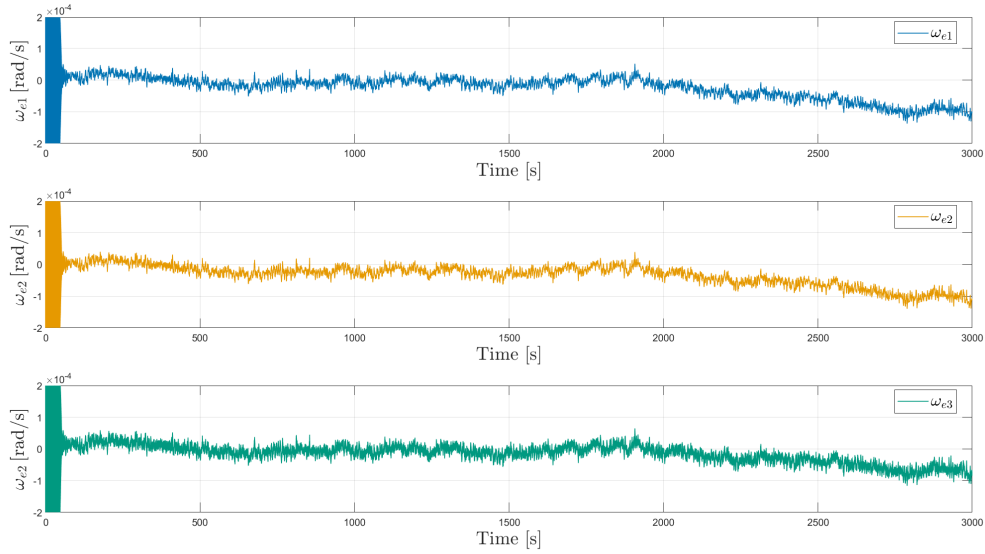


Figure 18: Time histories of the angular rates for 200 Monte-Carlo runs

The Monte Carlo results confirm that the proposed PD control law ensures acceptable pointing performance across a wide range of initial conditions. From a statistical perspective, the controller model can therefore be considered robust and validated for the considered nadir-pointing mission scenario.

2.2 Exercise 2: Software-In-the-Loop (SIL)

Once Model-In-the-Loop is completed, **prepare** your model setup to perform Software-In-the-Loop simulation (see lecture 12). Consider the following tasks to generate and test **C code** for the controller:

- 1) **Configure the simulation environment for SIL**

After a proper configuration of Simulink *Model Setting* (see lecture 12), make the *Controller Model* an atomic model and create its reference model.

- 2) **SIL Simulation**

Perform Model-In-the-Loop and Software-In-the-Loop simulations with Simulink SIL/PIL Manager. Note: SIL/PIL Manager is in the Simulink Apps.

- 3) **Results Comparison**

Compare the components of the reaction wheels control torque obtained with SIL against MIL results.

- 4) **Plot Comparison**

Generate one plot for each of the three components of the reaction wheel control torque. Each plot must display both the corresponding MIL and SIL simulation results.

Discussing the comparison results, are the MIL and SIL outputs consistent? Determine the maximum absolute difference (for each component) between the MIL and SIL outputs over the entire simulation. Based on these values, propose a reasonable numerical tolerance for validating MIL-SIL consistency. For each of the three components, plot the difference between the MIL and SIL simulation results (together with the chosen tolerance). What could explain those differences?

Software-In-the-Loop

The Software-In-the-Loop (SIL) simulation is performed to validate the C code automatically generated from the controller model and to assess its numerical consistency with the Model-In-the-Loop (MIL) implementation. The same simulation setup, initial conditions, solver configuration, and controller gains used in the MIL case are retained, so that any discrepancy between the two simulations can be attributed to code generation and numerical effects.

Results

Here, the results obtained through the SIL validation are reported.

Fig. 19 shows the comparison between the MIL and SIL time histories of the three components of the reaction wheel control torque. The two sets of signals are visually indistinguishable over the entire simulation time, indicating compliance between the MIL and SIL implementations. This confirms that the control logic, arithmetic operations and signal paths are preserved during the automatic C code generation process.

A more detailed quantitative comparison is provided in Fig. 20, which reports the absolute difference between the MIL and SIL torque commands for each reaction wheel. The observed differences remain several orders of magnitude smaller than the control torque levels of $\approx 10^{-6}$ and are bounded within a narrow range for the entire duration of the simulation.

These small discrepancies are mainly attributable to numerical effects due to probably the random noise applied to the sensors in the protected model or the error due to floating-point rounding, differences in execution order between interpreted and compiled code.

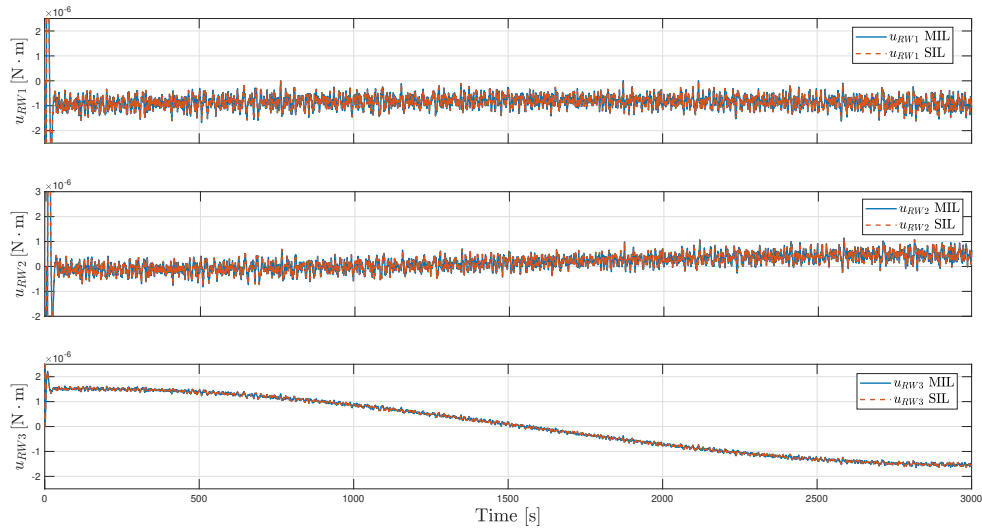


Figure 19: Comparison between MIL and SIL time histories of the three components of the reaction wheel control torque.

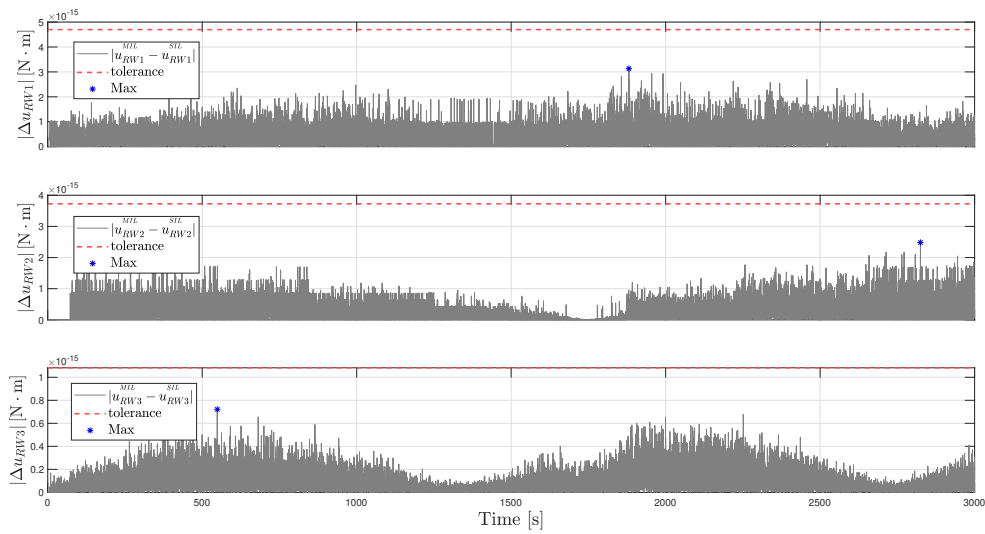


Figure 20: Absolute difference of each MIL and SIL time histories of the three components of the reaction wheel control torque.

To further quantify the consistency between the MIL and SIL simulations, the maximum absolute difference of the reaction wheel control torque is computed for each axis over the entire simulation time. The resulting values are reported in Table 3 and identified as blue * in Fig. 20. Based on the results of Table 3 a numerical tolerance of about $1.5 \times$ the maximum absolute difference, $< 10^{-14}$ N m magnitude, is considered reasonable for validating MIL–SIL consistency in this application. Within this tolerance, dashed red line Fig. 20, the MIL and SIL results are regarded as fully consistent.

Overall, the SIL analysis confirms that the automatically generated C code accurately reproduces the behavior of the MIL controller. This result validates the Software-In-the-Loop implementation and demonstrates that the control law is suitable for subsequent deployment.

Reaction wheel axis	Max. absolute difference [N·m]
$u_{RW,x}$	3.12×10^{-15}
$u_{RW,y}$	2.50×10^{-15}
$u_{RW,z}$	7.19×10^{-16}

Table 3: Maximum absolute difference between MIL and SIL reaction wheel control torque components.

2.3 Exercise 3: Processor-In-the-Loop (PIL)

After completing the MIL and SIL activities, set your model to perform a Processor-In-the-Loop simulation using the ESP32 board (see lectures 13 and 14). Consider the following tasks:

1. **Create two Simulink Models**

Create two separate models, one responsible for the Host PC side of the simulation and the other responsible for the ESP32 side. Name each file as `lastname123456_PCside.slx` and `lastname123456_ESP32side.slx`, respectively.

2. **Configure the ESP32-side model**

Insert the *Controller Model* block and configure the hardware settings for the ESP32 deployment. Add and set up the necessary blocks to establish a bi-directional WiFi TCP/IP connection between the ESP32 and the PC side, ensuring a **blocking mode** simulation. Show and justify the blocks used, highlighting important parameters set inside the most critical blocks.

3. **Configure the PC-side model**

Insert the protected model block and the subsystem block responsible for the pointing performance. Add and set up the necessary blocks to establish a bi-directional WiFi TCP/IP connection between the PC and the ESP32 side, ensuring a **blocking mode** simulation. Show and justify the blocks used, highlighting important parameters set inside the most critical blocks.

4. **Data Logging**

On the PC-side model, log the reaction wheel control torque command received from the ESP32.

After having completed the necessary setup, run and compare the results from the SIL to the PIL simulations (considering the nominal case previously defined in Table 2):

- 5) **Build, deploy and start the ESP32 controller**

Using the Simulink Hardware tab, build, deploy, and start the code on the ESP32, granting the board successfully connects to the configured target IP address.

- 6) **Start and profile the PC-side simulation**

Run the PC-side model using the Simulink Profiler: go to the Debug tab, Performance, Simulink Profiler. In the new Profiler tab, press Profile to start the PIL. Wait for the PIL simulation to finish, avoiding the saturation of the WiFi network during the simulation to obtain the best results. Since the PIL uses a WiFi TCP/IP connection, occasional communication delays may occur, leading to spikes in the received control torque to be applied by the reaction wheels. Given this setup, sporadic spikes in the received control torque are acceptable.

- 7) **Report the PIL execution time**

Using the Profiler Report, state the total time required to complete the PIL simulation.

- 8) **Plots and results comparison**

Generate plots for each of the three components of the reaction wheel control torque (input of the protected model), considering a comparison between SIL and PIL simulation results.

Discuss whether the results from SIL-PIL simulations are consistent. Determine the maximum absolute difference (for each component) between the SIL and PIL outputs over the entire simulation. Based on these values, propose a reasonable numerical tolerance for validating SIL-PIL consistency. For each of the three components, plot the difference between the SIL and PIL

simulation results (together with the tolerance accordingly chosen to include any spikes in the PIL outputs, if present). What could explain those differences? Additionally, if your simulation results include spikes, state how your chosen numerical tolerance would change if those spikes were not present.

If the communication between the PC side and the ESP32 were configured in non-blocking mode, how would the simulation results change? Describe which additional logic must be implemented on the PC side (such as using the status port from the TCP/IP receive block to detect stale data and hold the last valid sample through a zero-order hold), and discuss whether controller gain adjustments on the ESP32 side would be required under this mode. Does this guarantee that the controller will perform reliably in subsequent runs with different communication delays or slightly different initial conditions?

Note: for the results comparison, the Simulink Data Inspector tool is recommended. Ensure that the logged variables use consistent names across the SIL and PIL simulation comparisons (or rename them afterward).

Processor-In-the-Loop (PIL)

To perform the Processor-In-the-Loop (PIL) simulation using the ESP32 board, two separate Simulink models are developed: one executed on the ESP32 (ESP32-side model) and one running on the Host PC (PC-side model). The two models communicate through a bi-directional WiFi TCP/IP connection, enabling the exchange of sensor data and control commands in real time.

ESP32-side model configuration

On the ESP32-side model, shown in Fig. 21, the Controller Model is deployed and executed in the ESP-32 hardware accordingly.

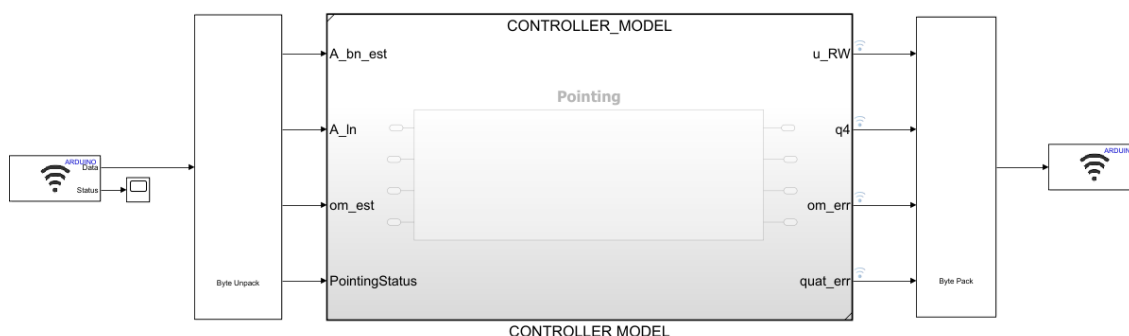


Figure 21: ESP32-side model

A blocking TCP/IP communication scheme is adopted to ensure synchronous data exchange between the ESP32 and the Host PC.

The bi-directional WiFi TCP/IP communication is implemented using two dedicated blocks: *WiFi TCP/IP Receive* and *WiFi TCP/IP Send*. The most relevant configuration parameters of these blocks are summarized in Table 4.

Parameter	Value
Data type	uint8
Data size (N)	172 bytes
Sample time	0.1 s

Table 4: WiFi TCP/IP Receive block parameters (ESP32-side).

The *WiFi TCP/IP Receive* block is used to receive sensor data from the PC-side model. The received TCP/IP packet is a one-dimensional vector of type `uint8`, in which the data transmitted correspond to A_{best} , A_{ln} , ω_{body} and a `PointingStatus` variable. The total data size is computed accordingly as

$$(9 + 9 + 3) \times 8 \text{ bytes} + 4 \text{ bytes.} \quad (16)$$

In the advanced settings, the *Wait until data received* option is enabled, and the timeout is set to 10s to compensate potential communication delays and allow the system to properly settle. The *Byte Unpack* block (refers to Table 5)) is then used to unpack the incoming byte stream into the physical quantities. The output port dimensions and data types are explicitly defined to match the expected controller inputs.

Parameter	Value
Output port dimensions	$\{[3, 3], [3, 3], [3, 1], [1]\}$
Output port data types	$\{\text{double}, \text{double}, \text{double}, \text{int32}\}$
Byte alignment	1

Table 5: Byte Unpack block parameters.

This configuration allows the correct reconstruction of the data required by the control law. After computing the control action, the ESP32 transmits the control-related signals back to the Host PC using the *WiFi TCP/IP Send* block. Both to the receive block and the send block are configured in *Server* mode and uses the same local IP port 25000 to maintain a communication channel. Before transmission, the controller outputs are sorted into a single byte stream using the *Byte Pack* block (refers to Table 6). This block packs multiple signals into a one-dimensional `uint8` vector, which is compatible with the TCP/IP communication interface.

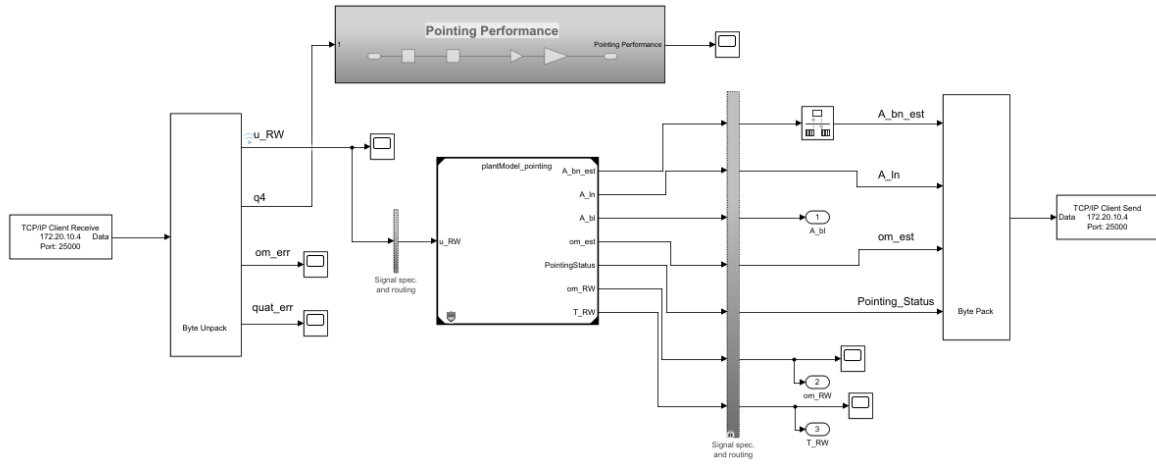
Parameter	Value
Output data type	uint8
Input port data types	$\{\text{double}, \text{double}, \text{double}, \text{double}\}$
Byte alignment	1

Table 6: Byte Pack block parameters.

The packed data vector includes the reaction wheel control torque u_{RW} , quaternion error information δq_4 , and angular velocity error ω_{err} .

PC-side model configuration

The PC-side model hosts the protected model and the the Pointing Performance subsystem (see section 2.1), as shown in Fig. 22.

**Figure 22:** PC-side model

This model guarantee the bi-directional WiFi TCP/IP communication of synchronous data exchange with the ESP32. Blocking mode is enabled for all the blocks discussed hereafter to guarantee that the controller is executed only after a complete data packet has been received from the ESP-32 side, ensuring temporal consistency between communication and control execution. This is enforced by setting the timeout of 10 s. The *TCP/IP Receive* block is used to receive control outputs data sent by the ESP32-side model, where its main parameters are reported in Table 7. The block is configured in client mode by specifying the remote IP address and port corresponding to the ESP32.

Parameter	Value
Remote IP address	172.20.10.4
Port	25000
Data size	88
Source data type	uint8
Block sample time	0.1 s

Table 7: TCP/IP Receive block parameters (PC-side).

The selected sample time is consistent with the controller execution rate. The data received from the ESP-32 are transmitted as a packed `uint8` vector. The *Byte Unpack* block (see Table 8) is therefore used to unpack the received data required by the protected model and by the Pointing Performance subsystem.

Parameter	Value
Output port dimensions	{[3, 1], [1], [3, 1], [1, 4]}
Output port data types	{double, double, double, double}
Byte alignment	1

Table 8: Byte Unpack block parameters (PC-side).

This configuration allows the correct reconstruction of the reaction wheel torque $\mathbf{u}_{RW}$, scalar pointing performance θ , angular velocity error ω_{err} , and quaternion error signals $\delta\mathbf{q}$. The data

size in Table 7 is accordingly computed as:

$$(3 + 1 + 3 + 4) \times 8 \text{ bytes} + 4 \text{ bytes.} \quad (17)$$

Furthermore, the *TCP/IP Send* block is used to transmit sensor estimates and state information from the protected model to the ESP32-side controller. The block is configured using the same IP address and port to maintain a consistent communication channel as reported in Table 9.

Parameter	Value
Remote IP address	172.20.10.4
Port	25000
Byte order	Little-endian

Table 9: TCP/IP Send block parameters (PC-side).

Moreover, the *Rate Transition* block is utilized to handle data transfer between different rates for A_{bnest} signal to be transmitted. Before transmission, the outputs of the protected model are sorted into a one-dimensional `uint8` vector using the *Byte Pack* block.

Parameter	Value
Output data type	uint8
Input port data types	{double, double, double, int32}
Byte alignment	1

Table 10: Byte Pack block parameters (PC-side).

The transmitted packet includes the estimated attitude matrix $\mathbf{A}_{\text{In}}$, angular velocity estimates ω_{est} , and the `PointingStatus` flag, data compliant with their data types reported in Table 10.

Results

Here, the results of the PC-side simulation are presented. From the Profiler Report the total execution time required to complete the PIL simulation is 3006.4s, which is close to the nominal simulation time $\text{simTime} = 3000\text{s}$. This result indicates that the introduction of the PIL architecture and the blocking TCP/IP communication does not introduce a significant computational overhead, and that the simulation proceeds in near real time with respect to the specified execution rate.

Fig. 23 shows the comparison between the SIL and PIL time histories of the three components of the reaction wheel control torque. The two sets of signals are in very good agreement and remain visually indistinguishable for the majority of the simulation time. This indicates that the control law executed on the ESP32 reproduces the SIL behavior with high fidelity.

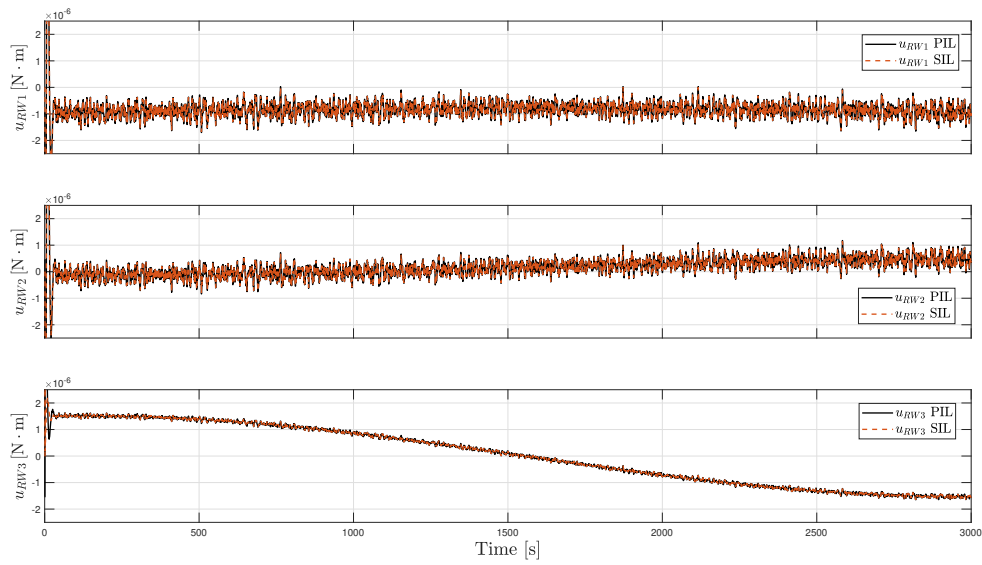


Figure 23: Comparison between PIL and SIL time histories of the three components of the reaction wheel control torque.

The absolute difference between the two signals is reported in Fig. 24 together with the selected numerical tolerance, enabling a direct assessment of the numerical consistency between the SIL and PIL results.

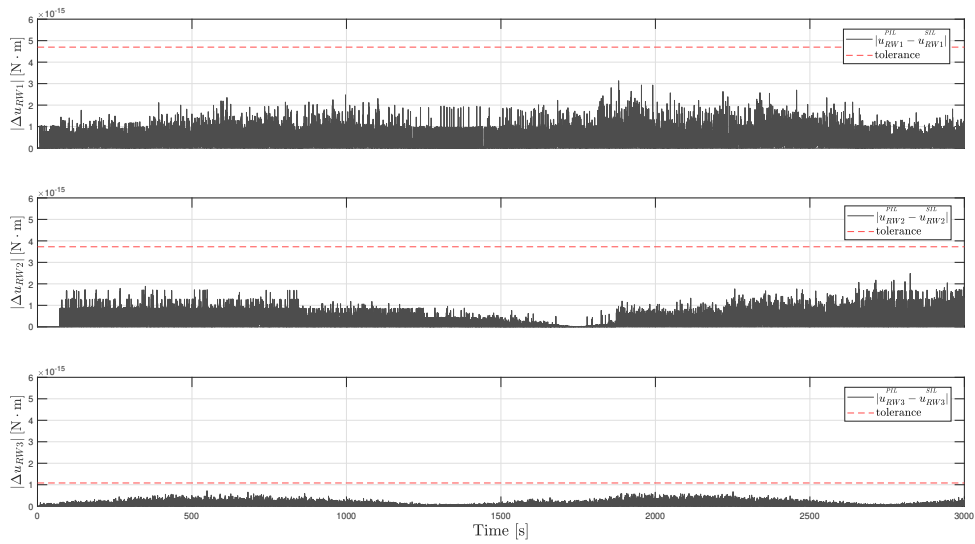


Figure 24: Absolute difference of each PIL and SIL time histories of the three components of the reaction wheel control torque

The maximum absolute differences observed for each reaction wheel torque component are reported in Table 11.

Reaction wheel axis	Max. absolute difference [N·m]
$u_{RW,x}$	3.12×10^{-15}
$u_{RW,y}$	2.51×10^{-15}
$u_{RW,z}$	8.10×10^{-16}

Table 11: Maximum absolute difference between PIL and SIL reaction wheel control torque components.

All discrepancies are on the order of $\approx 10^{-15}$ Nm, which is several orders of magnitude smaller than the nominal torque levels, typically ranging between 10^{-6} and 10^{-5} Nm. Based on these results, since no spikes are present an absolute tolerance in the order of 10^{-15} different for each component computed as $\times 1.5$ the maximum absolute difference.

It can be noted that the maximum discrepancies for the $u_{RW,x}$, $u_{RW,y}$ and $u_{RW,z}$ components are comparable to those observed in the MIL-SIL validation. This behavior needs further investigation, given the different computational platforms used in the MIL and PIL simulations. The extremely small differences observed indicate that the ESP32 implementation closely matches the numerical behavior of the Simulink controller, with residual deviations limited to floating-point round-off effects.

Finally, for what concerns the implications of using a non-blocking TCP/IP communication mode, in this case the simulation would no longer wait for new data to be received at each integration step. As a result, the PC-side model could occasionally operate on inaccurate data when a new TCP/IP packet has not yet arrived, effectively introducing variable delays in the control signal. This would lead to time intervals during which the same control torque sample is reused for multiple simulation steps.

To correctly handle this behavior, additional logic must be implemented on the PC side to be compliant to the *TCP/IP Receive* block. In particular, the status output of the receive block can be used to detect whether new data are available. When valid data are present, after being received and unpacked, are stored as the last valid sample. Although, in case of unavailable data, a zero-order hold mechanism is used to propagate the last valid sample to the rest of the model. This functionality can be implemented using a status-based *Switch* block combined with a *Unit Delay* or a *Zero-Order Hold* block, ensuring that missing or inaccurate data do not propagate through the simulation.

In other words, the non-blocking communication can be interpreted as introducing a small, bounded deviation from the ideal timing, which acts as a perturbation on the nominal control execution timing. However, this configuration does not guarantee deterministic or repeatable behavior across runs with different communication delays or slightly different initial conditions. Ensuring reliable performance under such conditions would require a dedicated stability analysis that explicitly accounts for time-varying communication delays. For these reasons, the blocking communication mode adopted in this work provides a more reliable and repeatable framework for Processor-In-the-Loop validation.

References

- [1] James R. Wertz. *Spacecraft Attitude Determination and Control*. Dordrecht: Springer, 1978.